ARCHITECTURE BRIEF

Nexus Learn Loop

Connected files, current data path, and next steps

Flask + Jinja templates | SQLite today | Supabase planned
Reviewed 20 October 2026

Bottom line: The app is a working Flask learning platform with modular blueprints, session login, quizzes, timer XP, leaderboard, and Gemini AI mentor. Live data still goes through **SQLite** (`database/database.py` + `schema.sql`). Supabase files exist as a starter kit only — they are not wired into `app.py` yet. Authentication is Flask sessions + password hashes in SQLite, not Supabase Auth.

1. How the project is wired today

Everything starts in **app.py**. It loads `.env`, sets the secret key, initializes SQLite, injects **current_user** into templates, and registers five blueprints. Templates render pages. JavaScript hydrates heatmaps, timer claims, resources search, LWB UI, and the AI mentor. Tests hit the same blueprints against a temporary SQLite file.

1.1 Runtime stack (live path)

Entrypoint	app.py	create_app(), session user, CORS, blueprint registration, SQLite init + seed
Auth pages	blueprints/auth.py → login.html, register.html	Register / login / logout. Passwords hashed with Werkzeug. Session stores user_id
Learner pages	blueprints/dashboard.py → index, dashboard, profile, leaderboard HTML	Landing, XP dashboard, profile editor, public rankings
Learning loop	blueprints/learning.py → quiz, result, timer, resources, lwb HTML	Server-scored quiz, Pomodoro XP, curated resources, LWB preview
AI mentor	blueprints/ai_mentor.py → ai_agent.html + ai_agent.js	Gemini chat via /api/ai-* using GEMINI_API_KEY
JSON APIs	blueprints/api.py ← main.js, timer.js	/api/current-user, leaderboard, record-quiz, record-study-session, activity-counts
Live database	database/database.py + schema.sql → instance/nexus.db	Users, profiles, XP log, study sessions, quiz results
Guards	services/security.py	login_required, CSRF, SECRET_KEY, safe CORS
Scoring	services/quiz_service.py, streak_service.py	Answer key + XP on the server; consecutive UTC-day streak

1.2 Page graph (templates connected by navigation)

These HTML files share one CSS file (`static/css/style.css`) and link to each other through Flask routes (and matching .html aliases).

index.html	login, register, dashboard, AI, quiz, timer, resources, leaderboard, LWB	Public home / CTAs
login.html ↔ register.html	dashboard after POST	Auth conversion
dashboard.html	AI, quiz, timer, leaderboard, resources, profile, LWB	Logged-in hub
quiz.html → result.html	dashboard, timer, leaderboard	Challenge loop
timer.html	dashboard (after POST XP), quiz, resources	Focus sprint
profile.html	dashboard, quiz, timer, leaderboard, login	Showcase + edit form
ai_agent.html	dashboard, resources, quiz, leaderboard, timer	Mentor workspace
lwb.html	dashboard, AI agent	Study-buddy preview (UI only)

1.3 Frontend scripts and the APIs they call

static/js/main.js	dashboard.html, profile.html	GET /api/activity-counts, GET /api/current-user
static/js/timer.js	timer.html	POST /api/record-study-session (JSON duration)
static/js/resources.js	resources.html	Client-only search/filter (no backend)
static/js/lwb.js	lwb.html	Local UI only — no WebRTC / Socket.IO yet
static/js/ai_agent.js	ai_agent.html	GET /api/ai-status, POST /api/ai-chat, /api/ai-models, /api/ai-test-key, or Gemini directly

1.4 Data writes (who saves what)

Register	POST /register	users + learner_profiles + learning_activity (+50 XP)
Login / logout	POST /login, POST /logout	session cookie only (no DB write)
Edit profile	POST /profile	learner_profiles + learning_activity (+15 XP)
Submit quiz	POST /quiz or POST /api/record-quiz	quiz_results + learning_activity (server XP)
Finish timer	POST /timer or POST /api/record-study-session	study_sessions + learning_activity
Dashboard / heatmap	GET /dashboard, GET /api/activity-counts	read-only aggregations
Leaderboard	GET /leaderboard, GET /api/leaderboard	SUM of learning_activity.xp_earned

2. SQLite vs Supabase — current truth

Supabase is not the live database yet. `app.py` always calls `database.database.init_db()`. Every blueprint does `import database.database as db`. USE_SUPABASE in `.env.example` is never read. `supabase` is not in `requirements.txt`. The adapter only implements stats read — not register, login, quiz write, or timer write.

database/schema.sql — integer IDs, users.password_hash	database/supabase_schema.sql — UUID user_id from auth.users, RLS policies
database/database.py — full CRUD used by all routes	database/supabase_client.py — client factory + supabase_get_user_stats only
instance/nexus.db created on first run	docs/supabase_guide.md + verify_supabase.py (script currently imports helpers that do not exist)
Tests run against temp SQLite	package.json lists @supabase/server (Node) — unused by the Flask app

SQLite tables (connected by user_id)

```
users 1■■1 learner_profiles
users 1■■* learning_activity (XP + heatmap dates)
users 1■■* study_sessions (timer minutes)
users 1■■* quiz_results (challenge scores)
leaderboard = GROUP BY users + SUM(xp_earned)
streak = distinct dates in learning_activity via streak_service.py
```

Supabase tables (designed, not connected)

```
auth.users (Supabase Auth) 1■■1 public.profiles
auth.users 1■■* public.learning_activity
auth.users 1■■* public.study_sessions
auth.users 1■■* public.quiz_results
public.leaderboard VIEW (needs extra SELECT policy for community ranks)
```

3. Gaps found in this review

- **Auth is local, not Supabase.** Register stores password_hash in SQLite. There is no JWT, no Google/GitHub OAuth, no auth.users trigger to create profiles.
- **Adapter is incomplete.** supabase_client.py does not implement create_user, get_user_by_login, record_quiz_result, record_study_session, get_leaderboard, or get_activity_counts.
- **verify_supabase.py is broken as written** — it imports get_supabase_url / get_supabase_key / get_supabase_anon_key, which are missing from supabase_client.py.
- **Leaderboard + RLS conflict:** activity rows are private (auth.uid() = user_id) while the leaderboard view needs public XP totals. That policy must be designed before go-live.
- **Integer IDs vs UUIDs:** Flask session currently stores integer user_id. Supabase Auth uses UUID. Session, APIs, and tests all need that change.
- **LWB is a mock UI.** lwb.js does not persist chat or voice. Needs Socket.IO / WebRTC later.
- **Only one live quiz** is served by the page (jinja2-interpolation); a second quiz exists in quiz_service.py but is unused.
- **package.json** has @supabase/server but the product is Python/Flask — do not mix that in unless you add a Node backend.

4. Next steps — authentication + Supabase as the database

Do this in order. Do not flip USE_SUPABASE=true until steps 1–5 work. Keep SQLite as a fallback until the adapter covers every function the blueprints already call.

Step 1 — Supabase project and schema

- Create a project at supabase.com. Copy Project URL, anon key, and service_role key into `.env` (never commit service_role).
- In SQL Editor, run `database/supabase_schema.sql`.
- Add a trigger: after insert on auth.users, insert a row into public.profiles (username from raw_user_meta_data).
- Fix/add RLS: public SELECT on leaderboard (or a SECURITY DEFINER view). Keep writes on activity/quiz/timer owner-only.
- Add `supabase` to `requirements.txt` and pip install it.

Step 2 — Finish the Python adapter (drop-in for database.py)

- Implement the same function names used by blueprints: create_user, get_user, get_user_by_login, update_profile, record_learning_activity, record_study_session, record_quiz_result, get_quiz_result, get_user_stats, get_leaderboard, get_activity_counts, seed_defaults_if_empty.
- Use the service_role key only on the Flask server. Never send it to the browser.
- Fix verify_supabase.py helpers (URL/key getters) so diagnostics actually run.
- Add a thin selector in `database/__init__.py`: if USE_SUPABASE=true import supabase adapter, else SQLite. Then app.py and blueprints stay unchanged.

Step 3 — Replace Flask password auth with Supabase Auth

- **Recommended:** Register/login call supabase.auth.sign_up / sign_in_with_password from blueprints/auth.py. Store the access token (or user UUID) in the Flask session. Stop writing password_hash locally.
- Optional later: Google / GitHub OAuth via Supabase Auth providers.
- Update login_required to treat a missing/expired Supabase user as logged out.
- On first login, ensure a profiles row exists (trigger or explicit insert).
- Award +50 XP signup bonus by inserting into learning_activity with the new UUID.
- Logout: session.clear() plus supabase.auth.sign_out() if you keep a refresh token server-side.

Step 4 — Wire writes (quiz, timer, profile, heatmap)

- Keep scoring in quiz_service.py (server still decides XP). Only the persistence target changes.
- Timer still validates 1–180 minutes in quiz_service; then insert study_sessions + learning_activity in Postgres.
- Heatmap GET /api/activity-counts should query learning_activity grouped by day for the session UUID.
- Leaderboard should read the Postgres view, not SQLite GROUP BY.

Step 5 — Tests, then cut over

- Keep existing unittest suite on SQLite (local CI).
- Add a small integration test (or verify_supabase.py) that inserts one profile, one quiz, one session, then reads stats.
- Set USE_SUPABASE=true only after register → dashboard → quiz → timer → heatmap → leaderboard all succeed against Postgres.
- Stop calling init_db() / seed_defaults against SQLite when Supabase is on (demo peers can be inserted once via SQL).

Step 6 — After the cutover (product, not blockers)

- Turn LWB from mockup into real rooms (Socket.IO + optional WebRTC).
- Serve more quizzes from the registry; persist question banks in Supabase if they grow.
- Move Gemini key fully server-side; stop encouraging client-side key paste in production.

- Deploy Flask (Render/Fly/Railway) with SECRET_KEY, SESSION_COOKIE_SECURE, and ALLOWED_ORIGINS.

5. Suggested file work list (implementation order)

1	database/supabase_schema.sql	Add auth.users → profiles trigger; leaderboard SELECT policy
2	database/supabase_client.py	Full CRUD matching database.py; URL/key helpers
3	database/__init__.py (new)	Switch SQLite vs Supabase from USE_SUPABASE
4	requirements.txt	Add supabase Python client
5	blueprints/auth.py	sign_up / sign_in_with_password; session user UUID
6	app.py + get_current_user	Skip SQLite init when USE_SUPABASE; UUID users
7	blueprints/api.py, dashboard.py, learning.py	No SQL change if adapter is drop-in
8	verify_supabase.py	Fix imports; optional write smoke test
9	tests/	Keep SQLite tests; add skipped-unless-env Supabase test
10	.env / .env.example	USE_SUPABASE=true only after Step 5

6. File inventory (all connected pieces)

app.py	security.py, database.py, all five blueprints, templates via context
blueprints/auth.py	database.py, security.py, login.html, register.html, dashboard
blueprints/dashboard.py	database.py, security.py, index/dashboard/profile/leaderboard.html
blueprints/learning.py	database.py, quiz_service.py, quiz/result/timer/resources/lwb.html
blueprints/ai_mentor.py	GEMINI_API_KEY, ai_agent.html, Google Generative Language API
blueprints/api.py	database.py, quiz_service.py, main.js, timer.js
database/database.py	schema.sql, streak_service.py, instance/nexus.db, every blueprint
database/supabase_client.py	streak_service.py, env vars — not imported by app.py
services/security.py	app.py, auth, dashboard, learning, api
services/quiz_service.py	learning.py, api.py
services/streak_service.py	database.py, supabase_client.py
static/js/*	matching templates + /api/* routes
tests/test_*.py	create_app + SQLite database.py
docs/supabase_guide.md	schema + env setup (documentation only)

Immediate next action: complete the Supabase adapter and Auth switch (steps 1–5). Until then, keep developing features against SQLite so tests stay fast. Do not treat LWB or extra Node packages as part of the database migration.